

DESARROLLO DE APLICACIONES WEB ENTORNO SERVIDOR:

DIARIO DE PRÁCTICAS- Semana 3

Alumno: Fara Santeyana, María Guillermina.

NIA: 13298631. **Curso:** 2do DAW.

Instituto: I.E.S La Sènia.

Año: 2025-2026.

	Descripción actividad	Orientaciones
23/03	QA con ejercicios grupales y presentación de herramienta interna	QA y herramienta interna
24/03	Testing con Gherkin + Behave. Con ejercicios prácticos de testing	BBD- gherkins- behave
25/03	Introducción a arquitectura hexagonal. Proyectos para reestructurar carpetas con tal arquitectura	Arquitectura Hexagonal
26/03	Domain Driven Design. Presentación de los principios y ejercicios aplicándolos	DDD
27/03	explicación de patrones y ejercicios sobre Value Objects y Agreggates	Value Objects y Agregattes. DDD

TESTING GHERKIN + BEHAVE

The screenshot displays a code editor with two files open. The left file, `api_steps.py`, contains Python code for Behave steps. It includes a `@step` decorator for sending a POST request, a `def` function `step_send_a_post_request`, and another `@step` decorator for sending a PUT request, followed by a `def` function `step_send_a_put_request`. The right file, `posts.feature`, is a Gherkin feature file. It starts with a `Feature: Posts API` and includes a `Background` section with a `Given` step for the API base URL. The main body contains two `Scenario` sections. The first scenario, 'Obtener todos los posts', has a `When` step for a GET request to `/posts`, followed by `Then` and `And` steps for status code 200 and a JSON array with more than 0 items. The second scenario, 'Obtener un post específico', has a `When` step for a GET request to `/posts/1`, followed by `Then` and `And` steps for status code 200 and specific fields in the response body. A third scenario, 'Crear un nuevo post', has a `When` step for a POST request to `/posts` with a body.

```
api.steps.py | posts.feature

61 # - Comparar context.response.status_code con status_code
62 # - Usar assert con mensaje descriptivo si falla
63 @step('the response status code is {status_code:d}') & mariafara
64 def step_verify_code_id(context, status_code):
65     actual_status = context.response.status_code
66     assert actual_status == status_code, (
67         f"Expected status code {status_code} but got {actual_status}. "
68         f"Response body: {context.response.text}"
69     )
70
71 # =====
72 # TODO: Implementar step para verificar que el body es un JSON array
73 # =====
74 # Pistas:
75 # - Usar context.response.json() para obtener el body como Python object
76 # - Verificar con isinstance(data, list)
77
78 @step('the response body is a JSON array') & mariafara
79 def step_verify_json_response(context):
80     data = context.response.json()
81     assert isinstance(data, list), (
82         f"Expected JSON array but got {type(data).__name__}. "
83         f"Response body: {data}"
84     )
85
86 # =====
87 # TODO: Implementar step para verificar longitud del array (mas de N)
88 # =====
89 # Pistas:
90 # - El decorador: 'the response array has more than {count:d} items'
91 # - Usar len() sobre context.response.json()
92 # - assert len(data) > count
93
94 @step('the response array has more than {count:d} items') & mariafara
95 def step_verify_array_len(context, count):
96     data = context.response.json()
97
98     # =====
99     # TODO: Implementar step para verificar longitud minima del array
100     # =====
101     # Pistas:
102     # - El decorador: 'the response array has at least {count:d} items'
103     # - Similar al anterior pero con >= en vez de >
104
105 @step('the response array has at least {count:d} items') & mariafara
106 def step_verify_array_has_at_least_items(context, count):
107     data = context.response.json()
108     assert len(data) >= count, (
109         f"Expected more than {count} items, but got {len(data)}"
110         f"Data : {data}"
111     )
112
113 # =====
114 # TODO: Implementar step para verificar longitud exacta del array
115 # =====
116 # Pistas:
117 # - El decorador: 'the response array length is {count:d}'
118 # - Comparar con == en vez de > o >=
119
120 @step('the response array length is {count:d}') & guillermina+1
121 def step_array_len_is(context, count):
122     data = context.response.json()
123     assert len(data) == count, (
124         f"Expected more than {count} items, but got {len(data)}"
125         f"Data : {data}"
126     )
127
128 # =====
129 # TODO: Implementar step para verificar que un campo existe en el body
```

```
Scenario: Crear un nuevo post
  When I send a POST request to "/posts" with body
  """
  {
    "title": "Hello world",
    "body": "first code execution",
    "userId": 1
  }
  """
  Then the response status code is 201
  And the response body contains field "id"
  And the response body field "title" equals "Hello world"

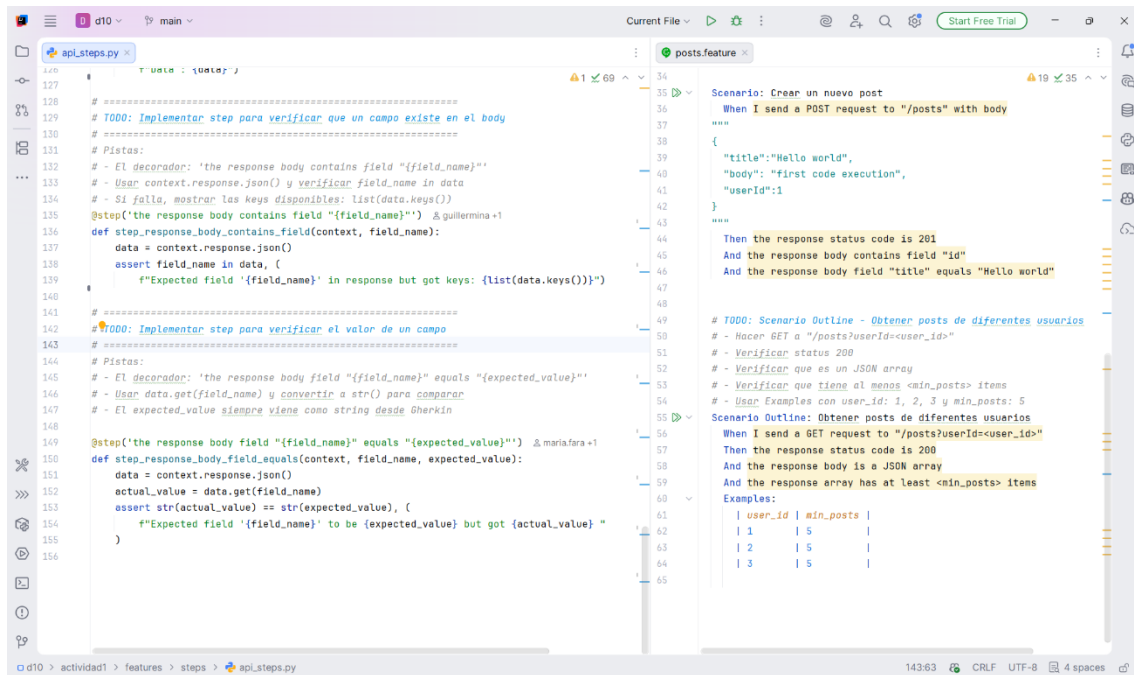
# TODO: Scenario Outline - Obtener posts de diferentes usuarios
# - Hacer GET a "/posts?userId=<user_id>"
# - Verificar status 200
# - Verificar que es un JSON array
# - Verificar que tiene al menos <min_posts> items
# - Usar Examples con user_id: 1, 2, 3 y min_posts: 5
Scenario Outline: Obtener posts de diferentes usuarios
  When I send a GET request to "/posts?userId=<user_id>"
  Then the response status code is 200
  And the response body is a JSON array
  And the response array has at least <min_posts> items
  Examples:
    | user_id | min_posts |
    | 1       | 5         |
    | 2       | 5         |
    | 3       | 5         |
```

```
api.steps.py | posts.feature

94 @step('the response array has more than {count:d} items') & mariafara
95 def step_verify_array_len(context, count):
96     data = context.response.json()
97     assert len(data) > count, (
98         f"Expected more than {count} items, but got {len(data)}"
99         f"Data : {data}"
100     )
101
102 # =====
103 # TODO: Implementar step para verificar longitud minima del array
104 # =====
105 # Pistas:
106 # - El decorador: 'the response array has at least {count:d} items'
107 # - Similar al anterior pero con >= en vez de >
108
109 @step('the response array has at least {count:d} items') & mariafara
110 def step_verify_array_has_at_least_items(context, count):
111     data = context.response.json()
112     assert len(data) >= count, (
113         f"Expected more than {count} items, but got {len(data)}"
114         f"Data : {data}"
115     )
116
117 # =====
118 # TODO: Implementar step para verificar longitud exacta del array
119 # =====
120 # Pistas:
121 # - El decorador: 'the response array length is {count:d}'
122 # - Comparar con == en vez de > o >=
123
124 @step('the response array length is {count:d}') & guillermina+1
125 def step_array_len_is(context, count):
126     data = context.response.json()
127     assert len(data) == count, (
128         f"Expected more than {count} items, but got {len(data)}"
129         f"Data : {data}"
130     )
131
132 # =====
133 # TODO: Implementar step para verificar que un campo existe en el body
```

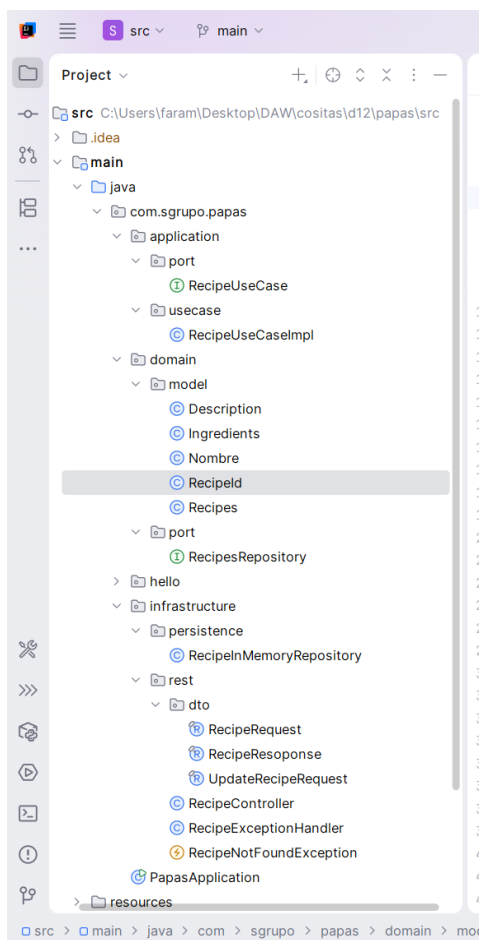
```
Scenario: Crear un nuevo post
  When I send a POST request to "/posts" with body
  """
  {
    "title": "Hello world",
    "body": "first code execution",
    "userId": 1
  }
  """
  Then the response status code is 201
  And the response body contains field "id"
  And the response body field "title" equals "Hello world"

# TODO: Scenario Outline - Obtener posts de diferentes usuarios
# - Hacer GET a "/posts?userId=<user_id>"
# - Verificar status 200
# - Verificar que es un JSON array
# - Verificar que tiene al menos <min_posts> items
# - Usar Examples con user_id: 1, 2, 3 y min_posts: 5
Scenario Outline: Obtener posts de diferentes usuarios
  When I send a GET request to "/posts?userId=<user_id>"
  Then the response status code is 200
  And the response body is a JSON array
  And the response array has at least <min_posts> items
  Examples:
    | user_id | min_posts |
    | 1       | 5         |
    | 2       | 5         |
    | 3       | 5         |
```



ARQUITECTURA HEXAGONAL

Inicio de proyecto individual. Primeros pasos en onion y hexagonal



DOMAIN

```
package com.sgrupo.papas.domain.model;

public class Nombre {
    private final String value;

    private Nombre(String value) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("The name field cannot be empty");
        }
        if (value.length() > 100) {
            throw new IllegalArgumentException("The name field cannot be longer than 100 characters");
        }
        this.value = value;
    }

    public static Nombre create(String value) { return new Nombre(value); }

    public String value() { return value; }

    @Override
    public int hashCode() { return value.hashCode(); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Nombre)) return false;
        return value.equals(((Nombre) o).value);
    }

    @Override
    public String toString() { return value; }
}

package com.sgrupo.papas.domain.model;

public class Description {
    private final String value;

    private Description(String value) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("The description field cannot be empty");
        }
        if (value.length() > 500) {
            throw new IllegalArgumentException("The description field cannot be longer than 500 characters");
        }
        this.value = value;
    }

    public static Description create(String value) { return new Description(value); }

    public String value() { return value; }

    @Override
    public int hashCode() { return value.hashCode(); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Description)) return false;
        return value.equals(((Description) o).value);
    }

    @Override
    public String toString() { return value; }
}

package com.sgrupo.papas.domain.model;

import java.util.UUID;

public class Recipes {
    private final RecipeId recipeId;
    private final Nombre nombre;
    private final Description description;
    private final Ingredients ingredients;

    private Recipes(RecipeId recipeId, Nombre nombre, Description description, Ingredients ingredients) {
        if (recipeId == null) {
            throw new IllegalArgumentException("The recipeId field must be valid");
        }
        if (nombre == null) {
            throw new IllegalArgumentException("The nombre field must be valid");
        }
        if (description == null) {
            throw new IllegalArgumentException("The description field must be valid");
        }
        if (ingredients == null) {
            throw new IllegalArgumentException("The ingredients field must be valid");
        }
        this.recipeId = recipeId;
        this.nombre = nombre;
        this.description = description;
        this.ingredients = ingredients;
    }

    public static Recipes create(RecipeId recipeId, Nombre nombre, Description description, Ingredients ingredients) {
        return new Recipes(recipeId, nombre, description, ingredients);
    }

    public void update(Nombre nombre, Description description, Ingredients ingredients) {
        if (nombre == null) throw new IllegalArgumentException("Nombre es obligatorio");
        if (description == null) throw new IllegalArgumentException("Description es obligatorio");
        if (ingredients == null) throw new IllegalArgumentException("Ingredientes son obligatorios");
        this.nombre = nombre;
        this.description = description;
        this.ingredients = ingredients;
    }
}

package com.sgrupo.papas.domain.model;

import java.util.UUID;

public class RecipeId {
    private final UUID value;

    private RecipeId(UUID value) {
        if (value == null) {
            throw new IllegalArgumentException("The id field must be valid");
        }
        this.value = value;
    }

    public static RecipeId create() {
        return new RecipeId(UUID.randomUUID());
    }

    public UUID value() { return value; }

    public static RecipeId of(UUID value) { return new RecipeId(value); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof RecipeId)) return false;
        return value.equals(((RecipeId) o).value);
    }

    @Override
    public int hashCode() { return value.hashCode(); }

    @Override
    public String toString() { return value.toString(); }
}
```

The screenshot shows an IDE with two files open: `Ingredients.java` and `RecipesRepository.java`. The `Ingredients.java` file is on the left, and the `RecipesRepository.java` file is on the right. The `Ingredients.java` file contains a `public class Ingredients` with a `private final List<String> items` and a `private Ingredients(List<String> items)` constructor. It also has a `public static Ingredients create(List<String> items)` method and a `public List<String> items()` method. The `RecipesRepository.java` file contains a `public interface RecipesRepository` with methods `void save(Recipes recipes)`, `Optional<Recipes> findById(RecipeId id)`, `List<Recipes> findAll()`, and `void deleteById(RecipeId id)`. The IDE interface includes a sidebar on the left with project navigation, a top bar with a 'Start Free Trial' button, and a bottom status bar showing the current file path and encoding.

APPLICATION

The screenshot shows an IDE with two files open: `RecipeUseCase.java` and `RecipeUseCaseImpl.java`. The `RecipeUseCase.java` file is on the left, and the `RecipeUseCaseImpl.java` file is on the right. The `RecipeUseCase.java` file contains a `public interface RecipeUseCase` with methods `RecipeId createRecipe(String name, String description, List<String> ingredients)`, `RecipeId getRecipe(String id)`, `List<Recipes> getAllRecipes()`, `Recipes updateRecipe(String id, String name, String description, List<String> ingredients)`, and `void deleteRecipe(String id)`. The `RecipeUseCaseImpl.java` file contains a `public class RecipeUseCaseImpl` that implements the `RecipeUseCase` interface. It has a `private final RecipeMemoryRepository repository` and implements the methods `createRecipe`, `getRecipe`, `getAllRecipes`, `updateRecipe`, and `deleteRecipe`. The IDE interface includes a sidebar on the left with project navigation, a top bar with a 'Start Free Trial' button, and a bottom status bar showing the current file path and encoding.

The screenshot shows an IDE with two files open: `RecipeUseCase.java` and `RecipeUseCaseImpl.java`. The `RecipeUseCase.java` file defines an interface with methods: `createRecipe`, `getRecipe`, `getAllRecipes`, `updateRecipe`, and `deleteRecipe`. The `RecipeUseCaseImpl.java` file implements these methods. It uses a `Repository` interface to interact with the data layer. The implementation includes logic for creating, finding, updating, and deleting recipes, with appropriate error handling for cases like 'Recipe not found'.

```
package com.sgrupo.papas.application.port;

import ...

//puerto de entrada
public interface RecipeUseCase {
    RecipeId createRecipe(String name, String description, List<String> ingredients);
    Recipes getRecipe(String id);
    List<Recipes> getAllRecipes();
    Recipes updateRecipe(String id, String name, String description, List<String> ingredients);
    void deleteRecipe(String id);
}

public class RecipeUseCaseImpl implements RecipeUseCase {
    public RecipeId createRecipe(String name, String description, List<String> ingredients) {
        return repository.createRecipe(name, description, ingredients);
    }

    @Override
    public Recipes getRecipe(String id) {
        return repository.findById(RecipeId.of(UUID.fromString(id)))
            .orElseThrow(() -> new RecipeNotFoundException("Recipe not found with id: " + id));
    }

    @Override
    public List<Recipes> getAllRecipes() {
        return repository.findAll();
    }

    @Override
    public Recipes updateRecipe(String id, String name, String description, List<String> ingredients) {
        Recipes recipe = repository.findById(RecipeId.of(UUID.fromString(id)))
            .orElseThrow(() -> new RecipeNotFoundException("Recipe not found with id: " + id));
        recipe.update(
            Nombre.create(name),
            Description.create(description),
            Ingredients.create(ingredients)
        );
        repository.save(recipe);
        return recipe;
    }

    @Override
    public void deleteRecipe(String id) {
        repository.deleteById(RecipeId.of(UUID.fromString(id)));
    }
}
```

INFRASTRUCTURE

The screenshot shows an IDE with two files open: `RecipeInMemoryRepository.java` and `RecipeController.java`. The `RecipeInMemoryRepository.java` file implements the `Repository` interface using an in-memory `HashMap`. It initializes a list of recipes and provides methods for saving, finding, and retrieving all recipes. The `RecipeController.java` file implements the `RecipeController` interface, handling HTTP requests for creating, finding, and updating recipes. It uses the `RecipeUseCase` interface to delegate the logic to the application layer.

```
@Repository
public class RecipeInMemoryRepository implements RecipesRepository {
    private Map<String, Recipes> databaseMap = new HashMap<>();

    public RecipeInMemoryRepository() {
        List<Recipes> initial = List.of(
            Recipes.create(
                RecipeId.create(),
                Nombre.create("Paella valenciana"),
                Description.create("La paella valenciana es un plato tradicional de la cocina de Valencia"),
                Ingredients.create(List.of("Arroz", "pollo", "conejo", "judías", "patatas"))
            ),
            Recipes.create(
                RecipeId.create(),
                Nombre.create("Tortilla de patatas"),
                Description.create("La tortilla de patatas, también conocida como tortilla española, es un plato típico de la cocina de España"),
                Ingredients.create(List.of("patatas", "huevos", "cebolla", "pimiento"))
            ),
            Recipes.create(
                RecipeId.create(),
                Nombre.create("Gazpacho"),
                Description.create("Sopa fría andaluza"),
                Ingredients.create(List.of("tomate", "pimiento", "pimiento"))
            )
        );
        initial.forEach(this::save);
    }

    @Override
    public void save(Recipes recipes) { databaseMap.put(recipes.getId().toString(), recipes); }

    @Override
    public Optional<Recipes> findById(RecipeId id) {
        return Optional.ofNullable(databaseMap.get(id.toString()));
    }

    @Override
    public List<Recipes> findAll() { return new ArrayList<>(databaseMap.values()); }
}

@RestController
@RequestMapping("/api/recipes")
public class RecipeController {
    private final RecipeUseCase recipeUseCase;

    public RecipeController(RecipeUseCase recipeUseCase) { this.recipeUseCase = recipeUseCase; }

    @PostMapping
    public ResponseEntity<RecipeResponse> create(@RequestBody RecipeRequest request) {
        RecipeId id = recipeUseCase.createRecipe(
            request.name(),
            request.description(),
            request.ingredients()
        );
        Recipes recipe = recipeUseCase.getRecipe(id.toString());
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(RecipeResponse.f(recipe));
    }

    @GetMapping
    public ResponseEntity<List<RecipeResponse>> getAll() {
        List<RecipeResponse> response =
            repository.findAll()
                .stream()
                .map(RecipeResponse::f)
                .toList();
        return ResponseEntity.ok(response);
    }

    @GetMapping("/{id}")
    public ResponseEntity<RecipeResponse> getById(@PathVariable String id) {
        Recipes recipe = recipeUseCase.getRecipe(id);
        return ResponseEntity.ok(RecipeResponse.f(recipe));
    }

    @PutMapping("/{id}")
    public ResponseEntity<RecipeResponse> update(@PathVariable String id, @RequestBody RecipeRequest request) {
        Recipes recipe = recipeUseCase.updateRecipe(id, request.name(), request.description(), request.ingredients());
        return ResponseEntity.ok(RecipeResponse.f(recipe));
    }
}
```

```
10 public class RecipeInMemoryRepository implements RecipesRepository {
11     public RecipeInMemoryRepository() {
12         recipeUseCase = new RecipeUseCase(
13             Nombre.create("Gazpacho"),
14             Description.create("Sopa fria andaluza"),
15             Ingredients.create(List.of("tomate", "pepino", "pimiento")
16         );
17     }
18     public void save(Recipes recipes) { databaseMap.put(recipes.getRecipeId(), recipes); }
19     public Optional<Recipes> findById(RecipeId id) {
20         return Optional.ofNullable(databaseMap.get(id.value().toString()));
21     }
22     public List<Recipes> findAll() { return new ArrayList<>(databaseMap.values()); }
23     public void deleteById(RecipeId id) {
24         if(!databaseMap.containsKey(id.value().toString())){
25             throw new RecipeNotFoundException("Recipe not found");
26         }
27         databaseMap.remove(id.value().toString());
28     }
29 }
30
31 public class RecipeController {
32     @GetMapping
33     public ResponseEntity<List<RecipeResponse>> getAll() {
34         List<RecipeResponse> response = recipeUseCase.getAllRecipes();
35         return ResponseEntity.ok(response);
36     }
37     @GetMapping("/{id}")
38     public ResponseEntity<RecipeResponse> getById(@PathVariable String id) {
39         Recipes recipe = recipeUseCase.getRecipe(id);
40         return ResponseEntity.ok(recipeUseCase.f(recipe));
41     }
42     @PutMapping("/{id}")
43     public ResponseEntity<RecipeResponse> update(@PathVariable String id, @RequestBody RecipeUpdate request) {
44         Recipes recipeUpdate = recipeUseCase.updateRecipe(id, request.name(), request.description(), request.ingredients());
45         return ResponseEntity.ok(recipeUseCase.f(recipeUpdate));
46     }
47     @DeleteMapping("/{id}")
48     public ResponseEntity<Void> delete(@PathVariable String id) {
49         recipeUseCase.deleteRecipe(id);
50         return ResponseEntity.noContent().build();
51     }
52 }
```

```
1 package com.sgrupo.papas.infrastructure.rest;
2
3 import ...
4
5 @RestControllerAdvice
6 public class RecipeExceptionHandler {
7     @ExceptionHandler({RecipeNotFoundException.class})
8     public ResponseEntity<String> handleRecipeNotFoundException(RecipeNotFoundException ex) {
9         return ResponseEntity.status(404).body(ex.getMessage());
10     }
11 }
12
13 package com.sgrupo.papas.infrastructure.rest;
14
15 public class RecipeNotFoundException extends RuntimeException {
16     public RecipeNotFoundException(String message) { super(message); }
17 }
```

The screenshot shows an IDE with three Java files open. The top two files, `RecipeRequest.java` and `UpdateRecipeRequest.java`, are side-by-side. The bottom file, `RecipeResponse.java`, is below them. The IDE interface includes a top bar with project information and a 'Start Free Trial' button, a left sidebar with icons for Explorer, Search, and Run and Debug, and a bottom status bar showing the file path and encoding.

```
1 package com.sgrupo.papas.infrastructure.rest.dto;
2
3 import java.util.List;
4
5 public record RecipeRequest (String name, String description, List<String> ingredients){
6 }
7
```

```
1 package com.sgrupo.papas.infrastructure.rest.dto;
2
3 import java.util.List;
4
5 public record UpdateRecipeRequest(String name, String description, List<String> ingredients){
6 }
7
```

```
1 package com.sgrupo.papas.infrastructure.rest.dto;
2
3 import ..
4
5
6
7 public record RecipeResponse(
8     String id, String nombre, String description, List<String> ingredients) {
9
10     @
11     public static RecipeResponse f(Recipes recipe){
12         return new RecipeResponse(recipe.getRecipeId().toString(), recipe.getNombre().value(), recipe.getDescription().value(), recipe.getIngredients().items());
13     }
14 }
```

papas > src > main > java > com > sgrupo > papas > infrastructure > rest > dto > RecipeResponse 5:23 CRLF UTF-8 4 spaces